



Lecture 9 — AI-Specific Architectural Patterns

Design patterns for RAG, agents, tool use, memory, multi-agent systems, edge AI and AI governance

- **CS455: Introduction to Software Engineering**

Learning objectives

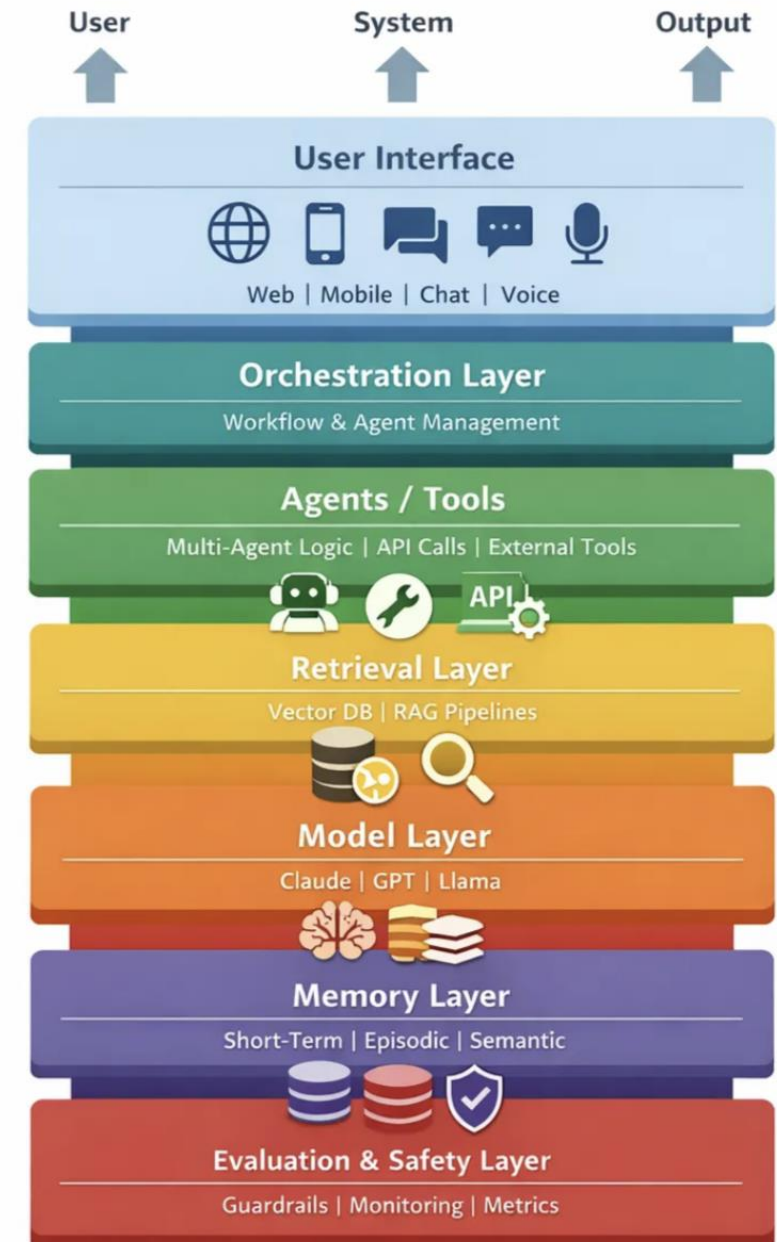
- Explain why AI systems motivate architectural patterns beyond ordinary application structure.
- Recognize common AI-specific patterns and the problem each pattern solves.
- Design RAG, tool-using agent, memory, human-in-the-loop and multi-agent architectures.
- Compare centralized, distributed, edge and hybrid AI deployment architectures.
- Identify architectural risks involving hallucination, model drift, prompt injection, cost, latency and tool misuse.
- Select and combine patterns according to system requirements.

Layered Pattern: Prompt Pipeline

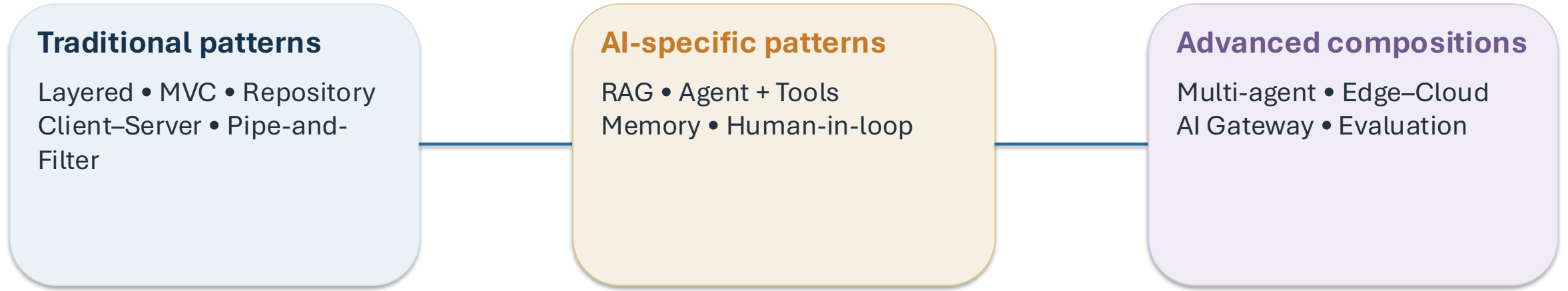
- **Input normalization:** Clean up user text
- **Context enrichment:** Add relevant info or history
- **Prompt templating:** Shape the prompt for the model
- **Model invocation**
- **Output post-processing**

Structured prompting dramatically reduces errors and inconsistency.

The Emerging LLM System Stack



1. From general patterns to AI-specific patterns



Key idea: AI-specific patterns do not discard software architecture; they specialize it for learned, probabilistic and context-dependent components.

2. Why do AI systems need specialized patterns?

The architecture must manage more than function calls

Uncertainty

Outputs may be wrong even when the software is functioning correctly.

Context

Good outputs may depend on retrieved documents, conversation state or tools.

Lifecycle

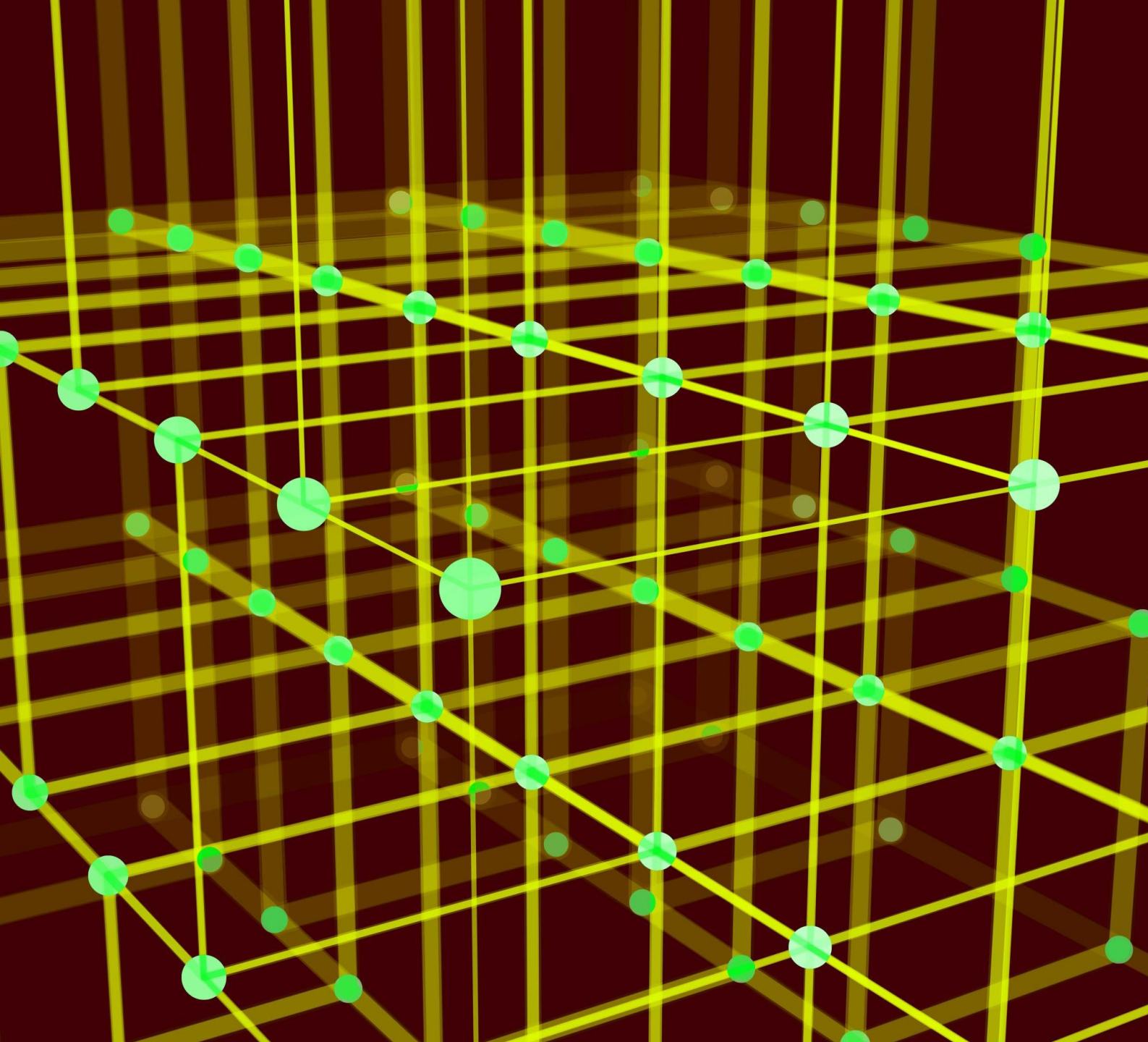
Models and data evolve independently of application code.

Autonomy

Agents can select actions dynamically instead of following one fixed workflow.

Risk

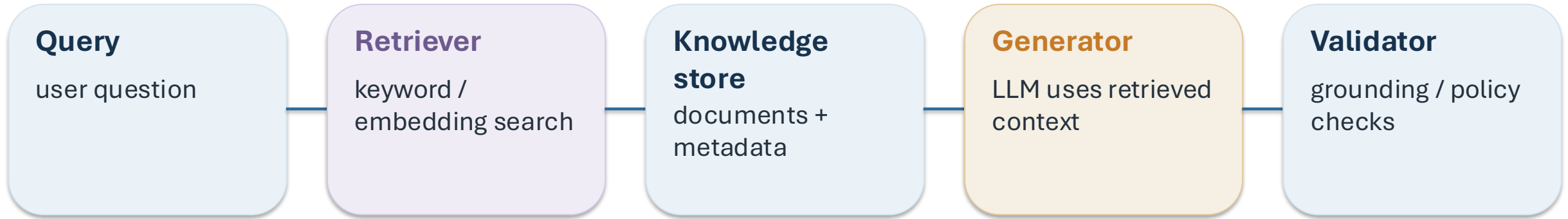
Prompt injection, data leakage, unsafe actions and bias require architectural controls.



AI Specific Patterns



3. Pattern: Retrieval-Augmented Generation (RAG) Separate knowledge retrieval from generation



- **Best for:** domain knowledge that changes frequently or should remain outside model parameters.
- **Key architectural issue:** retrieval quality can dominate answer quality.
- **Common controls:** metadata filtering, source attribution, re-ranking and answer validation.

4. RAG architecture variants

The pattern can be specialized

Basic RAG

retrieve top-k chunks → prompt LLM

Use when simple factual/domain QA

Hybrid RAG

keyword + vector retrieval

Use when mixed terminology + semantic similarity

Agentic RAG

agent decides when and what to retrieve

Use when retrieval decisions depend on the task

Graph RAG

retrieve entities/relations from a knowledge graph

Use when relationships matter more than isolated passages

Multi-stage RAG

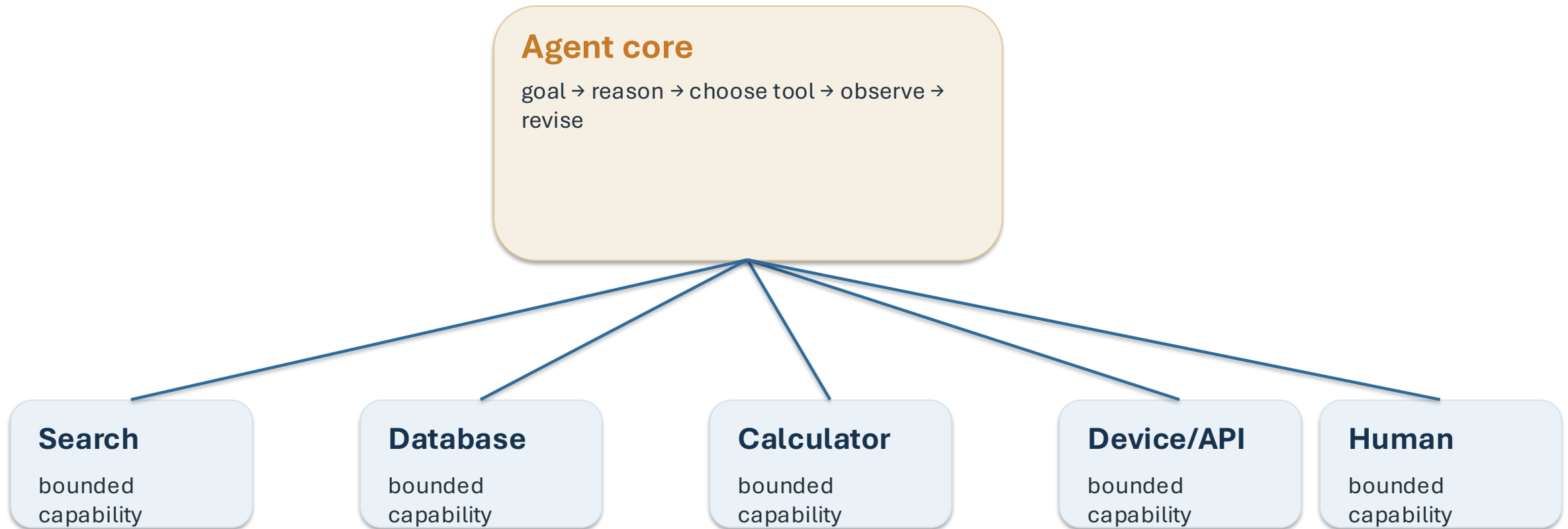
retrieve → re-rank → compress → generate

Use when long or noisy corpora require extra quality control

Pattern choice should follow the structure of the knowledge and the task—not the popularity of a particular implementation.

5. Pattern: Tool-Using Agent

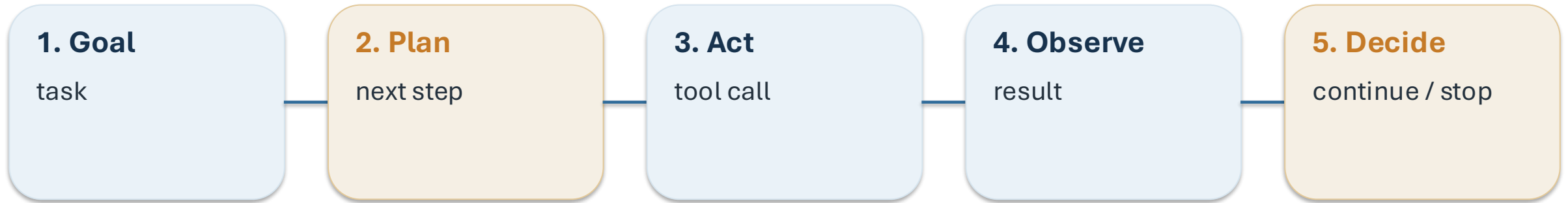
The model becomes a decision-maker over bounded capabilities



- Architectural principle: the agent should not have unrestricted access to arbitrary systems.
- Expose tools through explicit schemas, permissions, validation and audit logs.
- Use idempotent or reversible actions where possible.

6. Pattern: Agent Loop / ReAct-style control

Dynamic control flow replaces a fixed pipeline

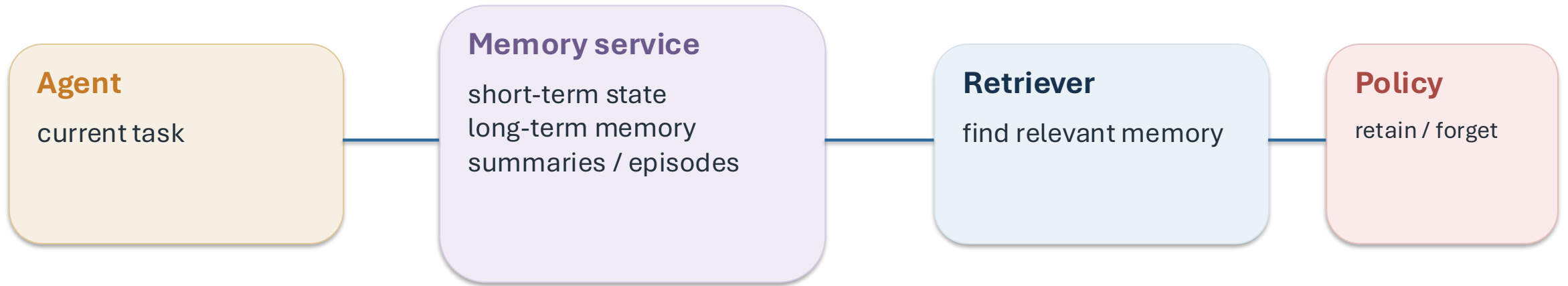


The loop can repeat until a stopping condition, budget or policy is reached.

- Architectural controls: maximum steps, time/token budget, tool permissions, state checkpoints and failure recovery.

7. Pattern: Externalized AI Memory

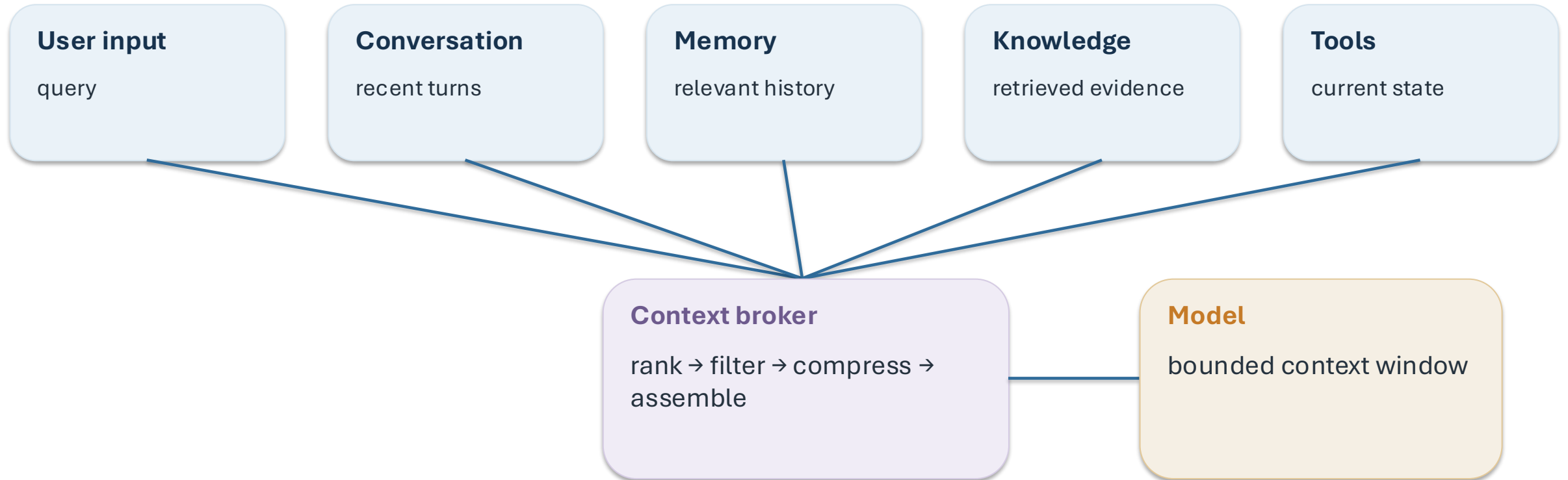
Persist selected state outside the model



- Do not store everything automatically: memory selection is an architectural policy.
- Important concerns: relevance, stale facts, privacy, retention, deletion and consistency.
- Memory can be implemented with databases, vector stores, event logs or combinations.

8. Pattern: Context Assembly / Context Broker

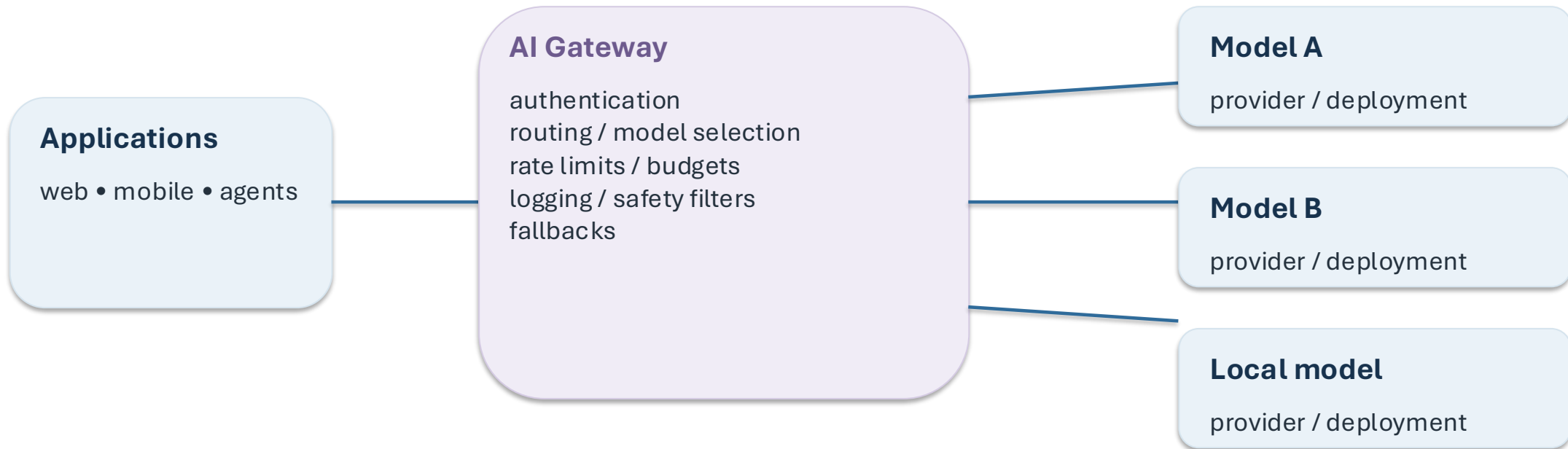
Treat model context as a managed architectural resource



- Why it matters: context windows are finite and irrelevant context can reduce quality.
- Architectural responsibility: decide what information is trusted, relevant and allowed to reach the model.

9. Pattern: AI Gateway

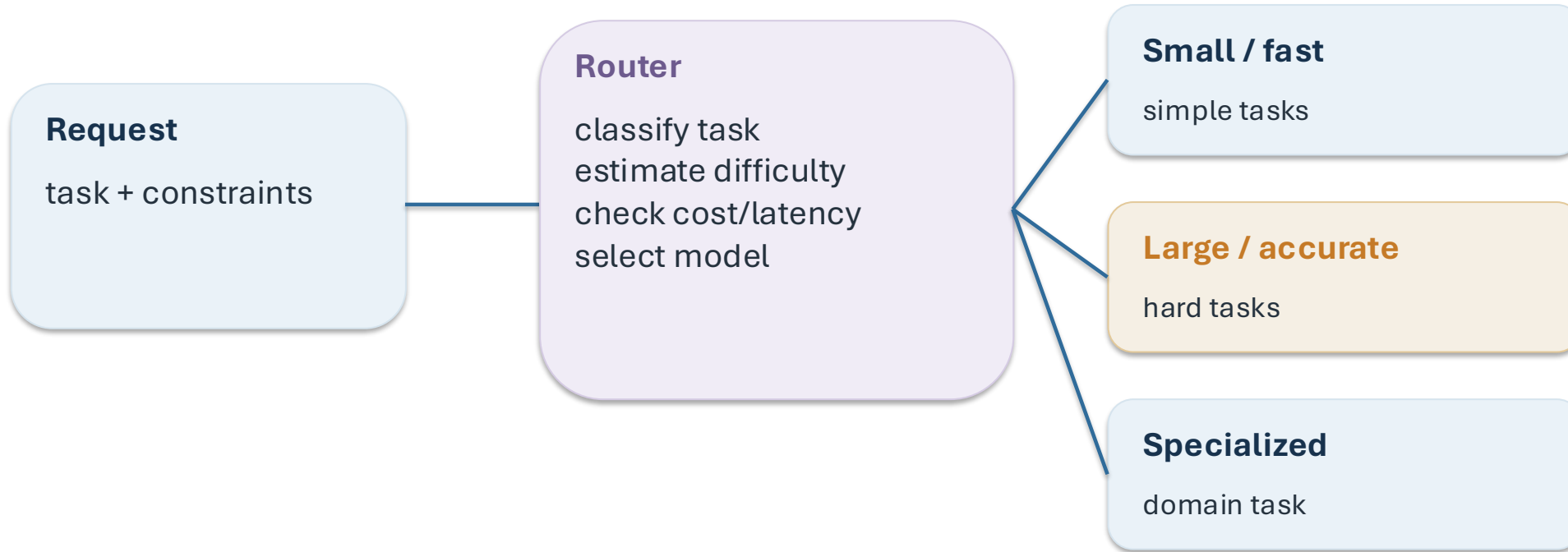
Centralize cross-cutting controls around model calls



- Useful when many applications share AI providers or when governance must be centralized.
- Trade-off: a gateway can become a bottleneck or single point of failure.

10. Pattern: Model Routing / Model Cascade

Use different models for different workloads



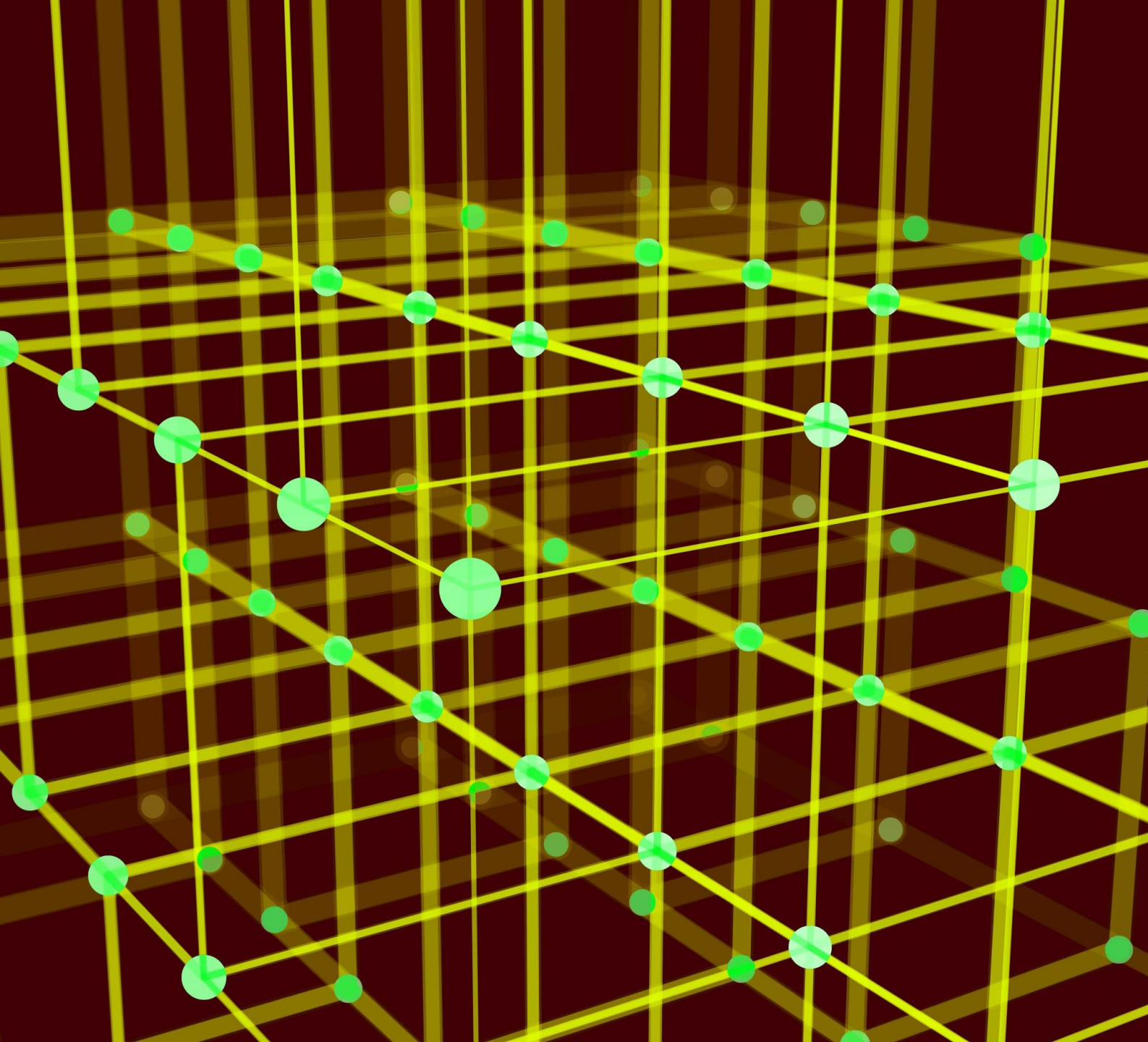
Goal: meet quality requirements while controlling latency, cost and resource use.

11. Pattern: Human-in-the-Loop / Human-on-the-Loop

Place people at explicit decision points



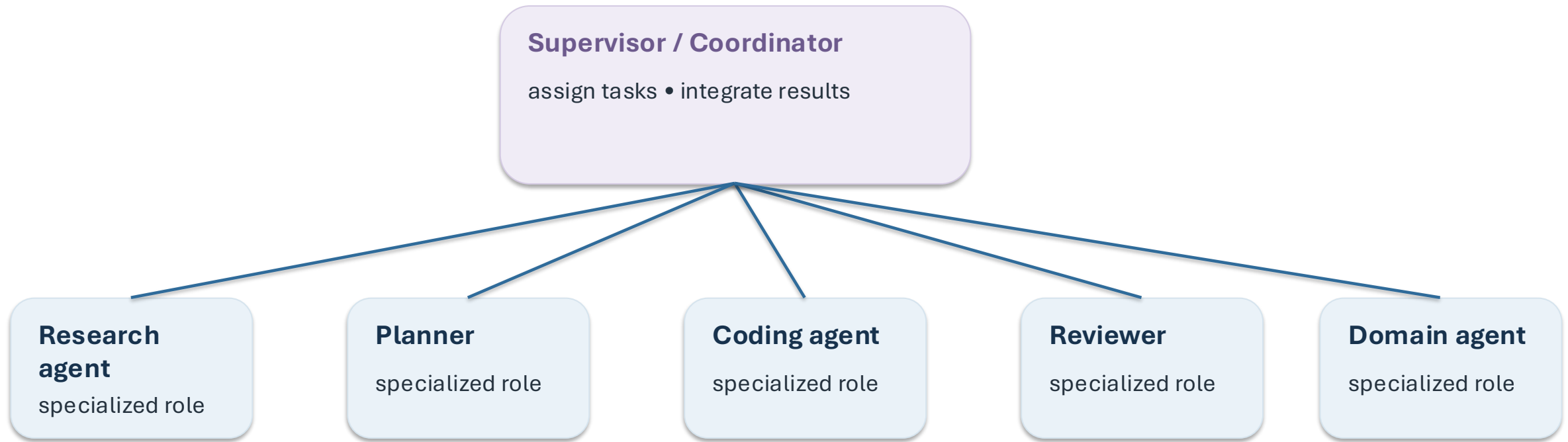
- Human-in-the-loop: person must approve before action.
- Human-on-the-loop: AI acts, but people supervise and can intervene.
- Use explicit escalation rules rather than assuming humans will notice failures.



Advanced
compositions

12. Pattern: Multi-Agent Architecture

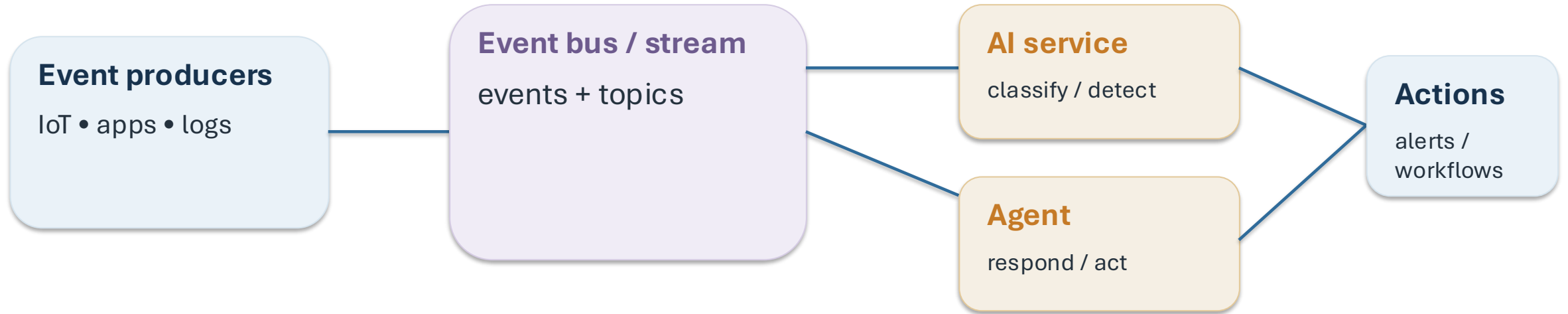
Decompose complex goals into specialized agents



- Benefits: separation of concerns, specialized prompts/tools and parallel work.
- Costs: coordination overhead, inconsistent state, more failure modes and higher cost.
- Do not use multiple agents when a simple deterministic workflow is sufficient.

13. Pattern: Event-Driven AI

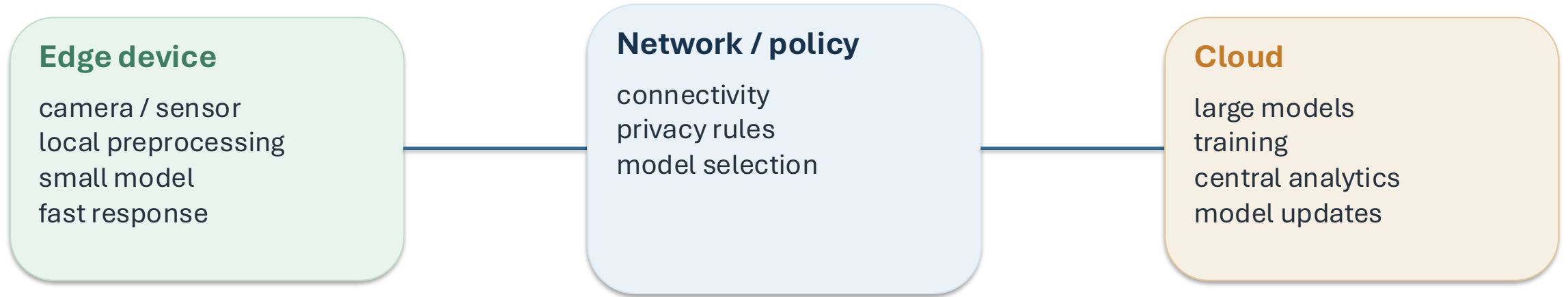
AI components react to events rather than only synchronous requests



- Useful for continuous monitoring, IoT, anomaly detection and asynchronous workflows.
- Important concerns: event ordering, duplicate events, backpressure and exactly-once vs at-least-once semantics.

14. Pattern: Edge–Cloud AI

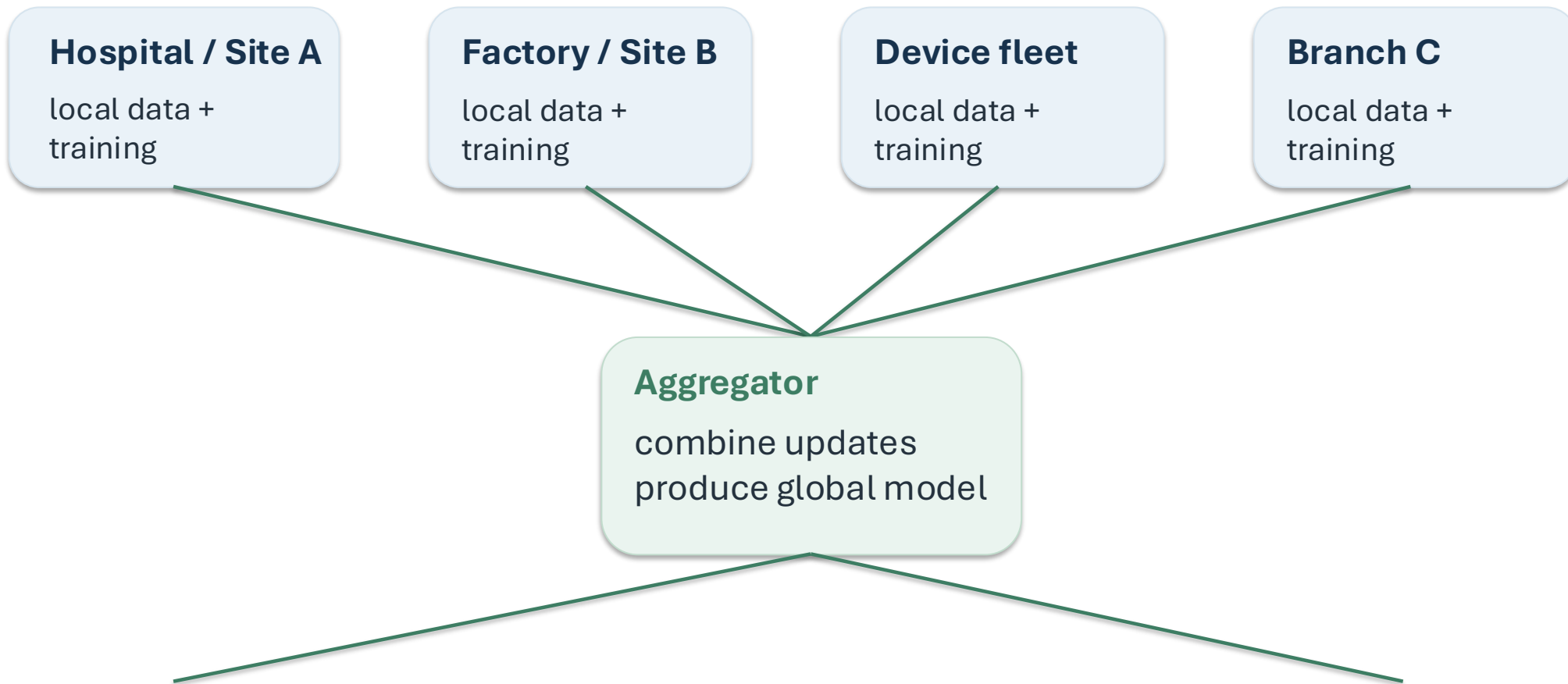
Move inference according to latency, privacy and resource constraints



- Edge advantages: low latency, reduced bandwidth and local data handling.
- Cloud advantages: compute scale, large models and centralized management.
- Hybrid systems can route only difficult or sensitive cases to the cloud.

15. Pattern: Federated / Privacy-Preserving AI

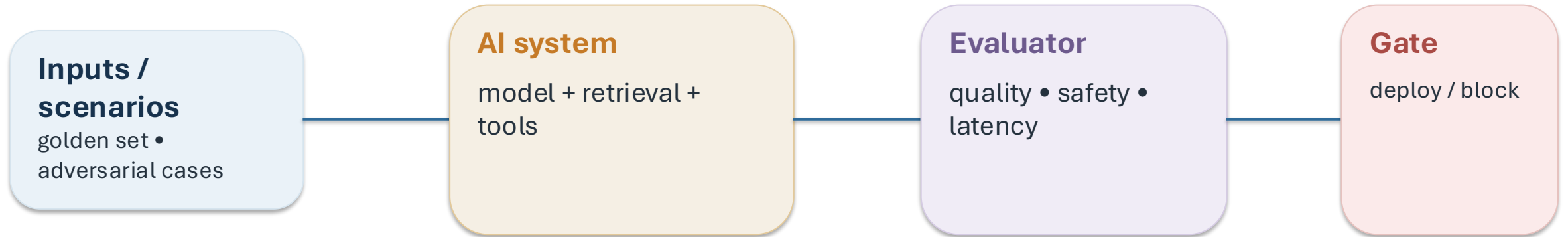
Learn from distributed data without centralizing all raw data



Architectural challenge: privacy, communication cost, heterogeneous data and trust among participants.

16. Pattern: Continuous AI Evaluation

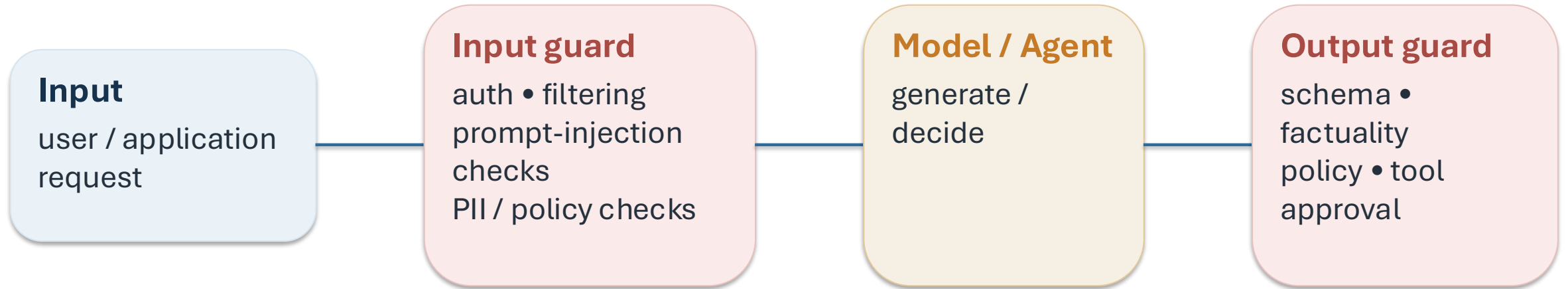
Testing the software is not enough



- Evaluate components and end-to-end behavior.
- Track regressions when prompts, models, retrieval indexes, tools or data change.
- Use offline evaluation before deployment and runtime monitoring after deployment.

17. Pattern: Guardrail / Policy Gateway

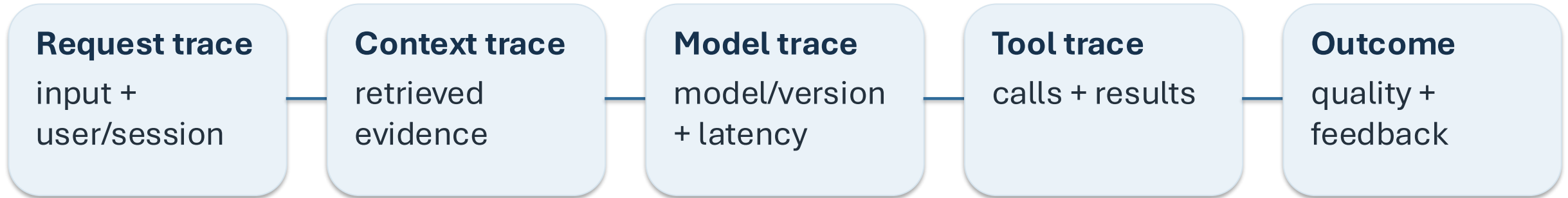
Put safety and governance controls around AI decisions



- Guardrails are architectural boundaries, not merely prompt wording.
- High-risk tools should have separate authorization and approval paths.
- Log policy decisions so failures can be investigated.

18. Pattern: AI Observability and Trace Architecture

Make AI behavior inspectable



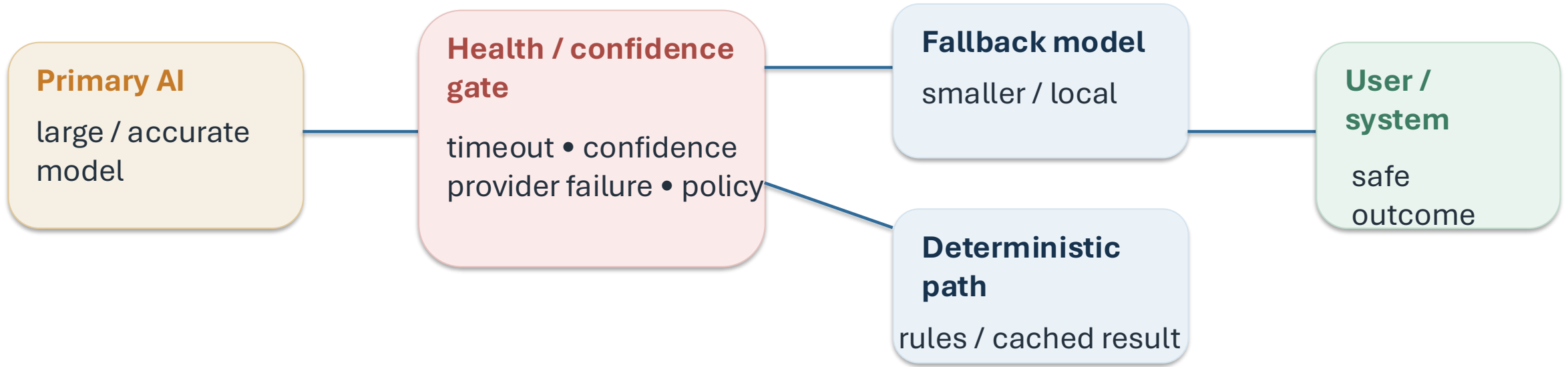
- Traditional logs tell us what code executed; AI traces should also explain what context and model version influenced a result.
- Traceability supports debugging, evaluation, cost analysis, compliance and incident response.

Example metrics

latency • token/compute cost • retrieval hit rate
• refusal rate • human correction rate

19. Pattern: Fallback and Graceful Degradation

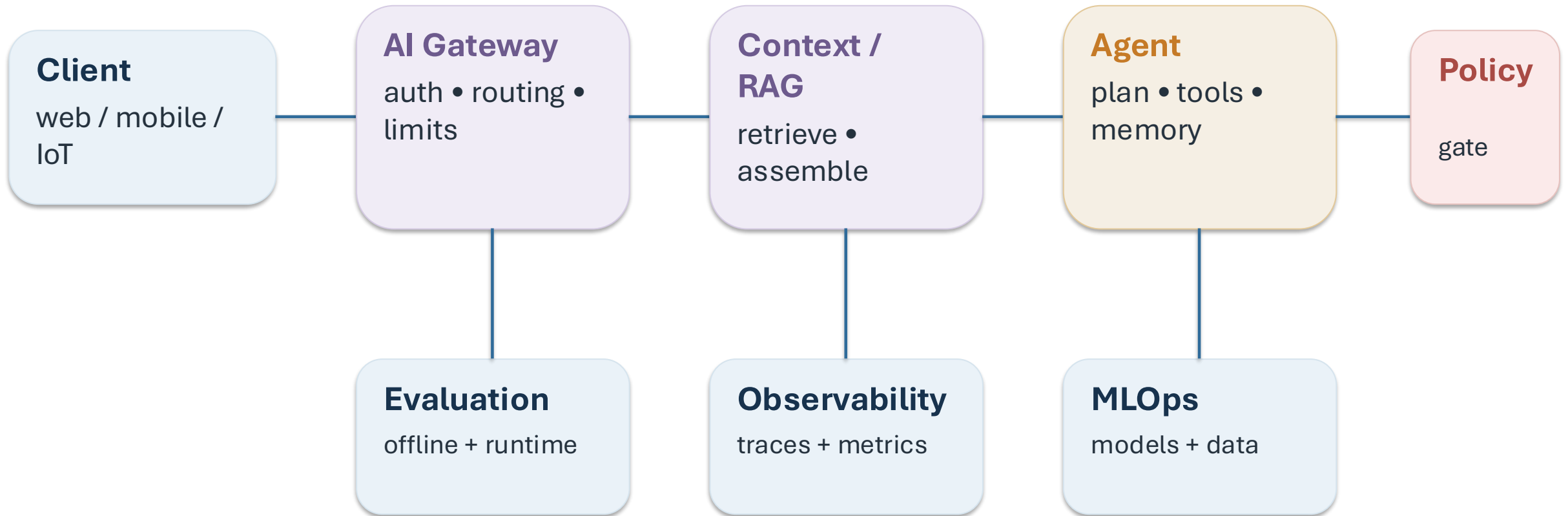
AI failures should not automatically become system failures



Design question: what is the safest useful behavior when the model is unavailable, uncertain or outside its domain?

20. Combining AI-specific patterns

Real systems usually use several patterns together



Patterns compose; they do not have to be implemented as separate products or services.

21. Pattern selection guide

Start with the problem, then choose the architecture

Need current external/domain knowledge?
RAG / Context Assembly

Need continuous reactions to events?
Event-Driven AI

Need the system to take actions?
Tool-Using Agent

Need local/low-latency inference?
Edge-Cloud AI

Need persistent task history?
Externalized Memory

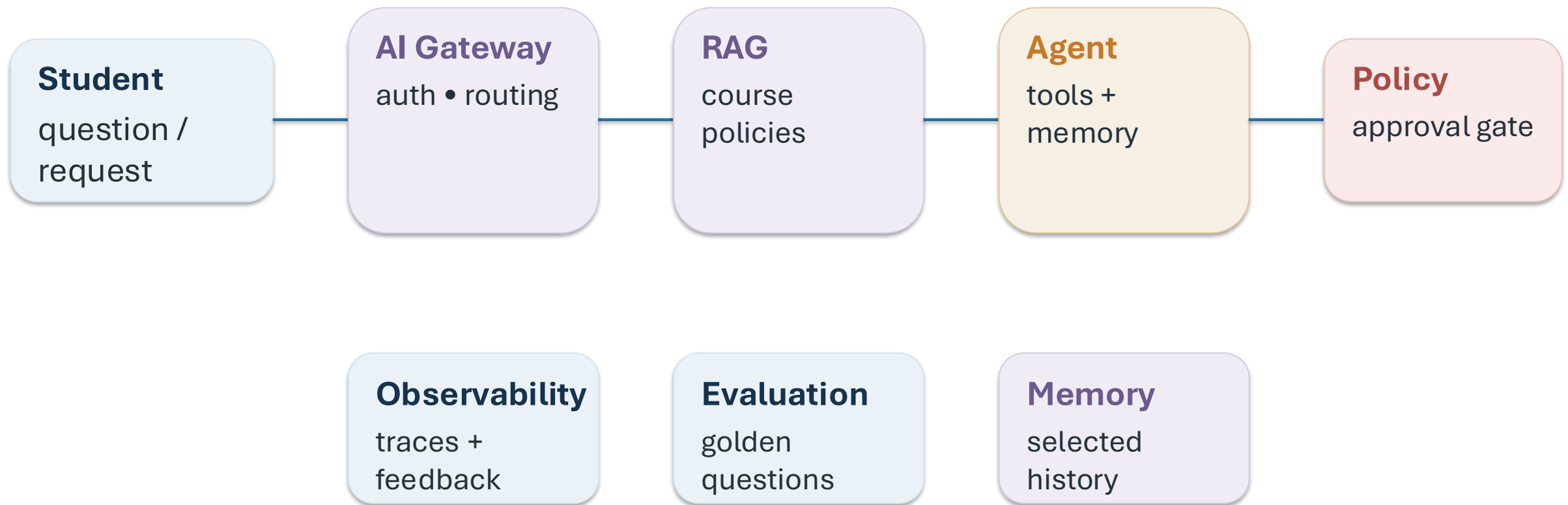
Need human approval for risky decisions?
Human-in-the-Loop + Guardrails

Need many model providers or models?
AI Gateway + Model Routing

Need specialized parallel roles?
Multi-Agent Architecture

22. Case study: AI assistant for university services

Apply multiple patterns



- Identify where hallucinations, stale policy information and unauthorized actions could occur.
- Then add one architectural control for each failure mode.

25. References

- Software architecture foundation: Ian Sommerville, Software Engineering, Chapter 6, Architectural Design.
- Zekeriya Beşiroğlu, “Architecture Patterns for LLM Systems” (Medium, Mar. 16, 2026): prompt pipelines, RAG,
- Yue Liu et al., “Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model Based Agents,” Journal of Systems and Software, vol. 220, 2025, 112278